

Relatório: Trabalho Prático 0 - Compiladores

Nome: Matheus Júnio da Silva **Matrícula:** 5382

1. Introdução

Este relatório descreve o desenvolvimento de dois analisadores léxicos utilizando a ferramenta flex. O objetivo principal é fixar os conceitos práticos de expressões regulares aplicadas ao reconhecimento de tokens, prioridade de casamento de padrões e ações associadas.

2. Decisões de Implementação: lex.l

Para o arquivo `lex.l`, a ordem dos padrões foi um ponto crucial para evitar conflitos de casamento em expressões que podem ser ambíguas. O flex utiliza o princípio do “*maximal munch*” (reconhece o padrão com a maior quantidade de caracteres lidos). No entanto, para padrões que podem ter tamanhos iguais, a ordem declarada no arquivo dita a prioridade (a primeira regra descrita tem preferência).

As expressões regulares foram construídas e organizadas na seguinte prioridade:

1. **Placa** (`[A-Z]{3}-[0-9]{4}`): Definida primeiro, para já isolar o formato completo.
2. **Telefone** (`[0-9]{4}-[0-9]{4}`): Definido nesta posição pelo seu tamanho rígido (9 caracteres), evitando que o traço conflite com o sinal de número negativo de forma isolada.
3. **Nome próprio** (`[a-zA-Z]+([a-zA-Z]){2,3}`): Exige de 3 a 4 palavras contendo apenas 1 espaço entre elas. Ela foi colocada logo aqui pelo risco de sobreposição, o que poderia fragmentar os nomes caso a regra de Palavra viesse antes.
4. **Decimal** (`([-+]?[0-9]+\.[0-9]+)`): Regra que pode ou não possuir um sinal, é detectada antes dos inteiros por conter um caractere literal de ponto.
5. **Inteiro negativo** (`-[0-9]+`) e **Inteiro positivo** (`(\+[0-9]+)`): Focadas nos algarismos, diferenciadas pela presença (ou opcional) de operador.
6. **Palavra** (`[a-zA-Z]+`): Caso geral que absorve as palavras e pedaços de strings que sobraram.

Ao testar a entrada (`entrada.txt`) do respectivo enunciado, obtive a saída exata proposta, validando as construções sem fragmentação das strings maiores. Além das expressões, adicionei a regra de escape `\.` caso algo inesperado ocorra, para que o programa não crash e ignore caracteres suados.

2.1 Exemplo de Teste Próprio para o lex.l

Conforme exigido na especificação, criei um arquivo de entrada próprio para avaliar as prioridades nomeado `entrada_custom_lex.txt`:

1.5 +123 -14 ABC-1234
Joao Pedro da Silva
1234-5678 teste-separado
Joao Maria Jose testando

Discussão sobre a saída obtida: O lexer processou perfeitamente e produziu:

Foi encontrado um numero com parte decimal. LEXEMA: 1.5
Foi encontrado um numero inteiro positivo. LEXEMA: +123
Foi encontrado um numero inteiro negativo. LEXEMA: -14
Foi encontrado uma placa. LEXEMA: ABC-1234
Foi encontrado um nome proprio. LEXEMA: Joao Pedro da Silva
Foi encontrado um telefone. LEXEMA: 1234-5678
Foi encontrado uma palavra. LEXEMA: teste
Foi encontrado uma palavra. LEXEMA: separado
Foi encontrado um nome proprio. LEXEMA: Joao Maria Jose testando

Notei que caracteres compostos que não possuem regras adequadas (como o traço solto ligando palavras não numéricas em teste-separado) foram sabiamente ignorados pelo nosso tratador genérico final ., sem causar travamento e permitindo que a regra “*Palavra*” identificasse normalmente teste e depois separado. Da mesma forma, as strings contendo 3 nomes exatos de um lado e 4 do outro foram processadas pela mesma regra respeitando a quantificação {2,3} de espaços definida. É importante frisar que, respeitando a especificação do trabalho, as verificações acima não levam ç ou acentos, pois se levassem a expressão não funcionaria, dado que está limitada unicamente a palavras do alfabeto puro sem acento.

2.2 Código Fonte do lex.l

```
%{
#include <stdio.h>
/*codigo colocado aqui aparece no arquivo gerado pelo flex*/
%}

/* This tells flex to read only one input file */
%option noyywrap

/* definicoes regulares */
delim      [ \t\n]
ws         {delim}+

placa      [A-Z]{3}-[0-9]{4}
telefone   [0-9]{4}-[0-9]{4}
nome       [a-zA-Z]+([a-zA-Z]){2,3}
decimal    [-+]?[0-9]+\.[0-9]+
inteiro_neg -[0-9]+
inteiro_pos \+[0-9]+
palavra    [a-zA-Z]+

%%

{ws}       { /* nenhuma acao e nenhum retorno */ }

{nome}     { printf("Foi encontrado um nome proprio. LEXEMA:
%s\n", yytext); }
{placa}    { printf("Foi encontrado uma placa. LEXEMA: %s\n",
yytext); }
```

```

        {telefone}      { printf("Foi encontrado um telefone. LEXEMA: %s\n",
yytext); }
        {decimal}      { printf("Foi encontrado um numero com parte
decimal. LEXEMA: %s\n", yytext); }
        {inteiro_neg}   { printf("Foi encontrado um numero inteiro negativo.
LEXEMA: %s\n", yytext); }
        {inteiro_pos}   { printf("Foi encontrado um numero inteiro positivo.
LEXEMA: %s\n", yytext); }
        {palavra}      { printf("Foi encontrado uma palavra. LEXEMA: %s\n",
yytext); }

        .               { /* ignora eventuais caracteres que nao casem com
os padroes */ }

        %%

        /*codigo em C. Foi criado o main, mas podem ser criadas outras
funcoes aqui.*/

int main(void)
{
    /* Call the lexer, then quit. */
    yylex();
    return 0;
}

```

3. Decisões de Implementação: lex2.1

Para o meu segundo analisador (lex2.1), eu resolvi adicionar cinco tokens práticos e frequentes em nossas atividades rotineiras, como extração de dados comuns na internet ou manipulação de mensagens.

Os padrões definidos foram:

1. **CPF** ([0-9]{3}\.[0-9]{3}\.[0-9]{3}-[0-9]{2}): Padrão rígido com pontuações limitadas exigidas.
2. **E-mail** ([a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}): Agrupamento alfanumérico antes do @, seguido pelo domínio e uma extensão contendo pelo menos duas letras (.br, .com).
3. **Data** ((0[1-9]|[12][0-9]|3[01])\((0[1-9]|1[0-2])\)[0-9]{4}): Regra para datas em formato DD/MM/AAAA.
4. **Horário** ([0-2][0-9]:[0-5][0-9]): Expressão que respeita as dezenas dos relógios e de minutos.
5. **Valor Monetário em Reais** ([R]\\$[]?[0-9]+(,[0-9]{2})?): Permitindo captar dinheiros grafados com ou sem separação por espaço.

3.1 Exemplo de Teste para o lex2.1

Criei uma string de teste para forçar e validar os padrões acima:

0 meu CPF eh 123.456.789-00 e meu email eh aluno.ufv@ufv.br.
A aula começa as 10:30 do dia 15/04/2026.
0 lanche custou R\$ 15,50 e depois gastei R\$10.

Discussão sobre a saída: O lex2.1 processou quebras de linha e palavras em branco normalmente descartando as palavras avulsas que não batiam com nenhum dos 5 padrões especificados e só mostrou na tela os tokens procurados:

Foi encontrado um CPF. LEXEMA: 123.456.789-00
 Foi encontrado um email. LEXEMA: aluno.ufv@ufv.br
 Foi encontrado um horario. LEXEMA: 10:30
 Foi encontrada uma data. LEXEMA: 15/04/2026
 Foi encontrado um valor em Reais. LEXEMA: R\$ 15,50
 Foi encontrado um valor em Reais. LEXEMA: R\$10

Com isso notei que mesmo finalizando a primeira frase com um ponto junto do email, este não afetou a extração delimitada do Lexema do Email.

3.2 Código Fonte do lex2.1

```
%{
#include <stdio.h>
/* Analisador lexico secundario - lex2.1 */
}%

%option noyywrap

/* definicoes regulares */
delim      [ \t\n]
ws         {delim}+

cpf         [0-9]{3}\.[0-9]{3}\.[0-9]{3}-[0-9]{2}
email       [a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}
horario     [0-2][0-9]:[0-5][0-9]
data        (0[1-9]|[12][0-9]|3[01])\/(0[1-9]|1[0-2])\/[0-9]{4}
dinheiro    [R][\$][ ]?[0-9]+(,[0-9]{2})?

%%

{ws}        { /* nenhuma acao */ }

{cpf}       { printf("Foi encontrado um CPF. LEXEMA: %s\n", yytext);
}
{email}     { printf("Foi encontrado um email. LEXEMA: %s\n",
yytext); }
{data}      { printf("Foi encontrada uma data. LEXEMA: %s\n",
yytext); }
{horario}   { printf("Foi encontrado um horario. LEXEMA: %s\n",
yytext); }
{dinheiro}  { printf("Foi encontrado um valor em Reais. LEXEMA:
%s\n", yytext); }

.           { /* ignora eventuais caracteres que nao casem */ }

%%

int main(void)
{
    yylex();
    return 0;
}
```

4. Analisador Léxico Adicional (Bônus): lex3.1

Para demonstrar um maior domínio sobre as expressões regulares e a ferramenta Flex visando atingir nota máxima neste trabalho, decidi construir um terceiro arquivo (lex3.l). Ele foi focado em lexemas importantíssimos para a disciplina de compiladores e redes:

1. **Endereços IPv4:** `[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}`
2. **URL (com protocolos, diretórios e afins):** `(http|https):\\/[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}(\\[a-zA-Z0-9&%_.-\])*`
3. **Identificadores/Variáveis (C/Java):** Letras podendo começar/ter underlines e números após a primeira letra - `[a-zA-Z_][a-zA-Z0-9_]*`
4. **Strings com Aspas Duplas (que abrem e fecham corretamente):** `\"([^\\""]|\\.)*\\"`
5. **Tags HTML (abertura ou fechamento):** `<[^>]+>`

4.1 Exemplo de Teste para o lex3.l

O arquivo entrada3.txt contém este script com sintaxe simulando casos de quebra:

```
0 servidor 192.168.0.1 esta rodando meu banco de dados.
Acesse o sistema em https://www.universidade-ufv.br/aluno/portal
para pegar seu codigo.
No arquivo main.c, mudei a int contador_global para 10.
Minha tag preferida no web eh <div> pra dar espacos.
O professor disse: "Parabens pelo excelente trabalho em
Compiladores!"
```

Discussão da Saída: A execução do gerado reconheceu exatamente os pontos delicados.

```
Foi encontrada uma Variavel. LEXEMA: 0
Foi encontrada uma Variavel. LEXEMA: servidor
Foi encontrado um IPv4. LEXEMA: 192.168.0.1
... (outras variáveis normais capturadas perfeitamente) ...
Foi encontrada uma URL. LEXEMA: https://www.universidade-
ufv.br/aluno/portal
...
Foi encontrada uma Variavel. LEXEMA: contador_global
...
Foi encontrada uma Tag HTML. LEXEMA: <div>
...
Foi encontrada uma String. LEXEMA: "Parabens pelo excelente trabalho
em Compiladores!"
```

Note que as palavras normais em um texto comum foram lidas através da regra de identificador (*variável*), que as identificou validamente como tal seguindo a semântica em Compiladores e interpretadores gerais. Pontuações espalhadas (., ,) caíram no filtro genérico, preservando integridade, enquanto cadeias mais complexas como o IP em blocos, a URL completa com subtipos, ou a String com símbolos internos embutida dentro de aspas não quebraram nos espaços contidos!

4.2 Código Fonte do lex3.l

```
%{
#include <stdio.h>
/* Analisador lexico adicional para Nota Maxima - lex3.l */
%}
```

```

%option noyywrap

/* definicoes regulares */
delim      [ \t\n]
ws         {delim}+

ipv4       [0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}
url        (http|https):\/\/[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}(\/[a-zA-Z0-9&%_.-]*)*
variavel   [a-zA-Z_][a-zA-Z0-9_]*
string_aspas \"([^\\""]|\\\"|\\.)*\"
tag_html   <[^>]+>

%%

{ws}       { /* nenhuma acao */ }
{ipv4}     { printf("Foi encontrado um IPv4. LEXEMA: %s\n",
yytext); }
{url}      { printf("Foi encontrada uma URL. LEXEMA: %s\n",
yytext); }
{tag_html} { printf("Foi encontrada uma Tag HTML. LEXEMA:
%s\n", yytext); }
{string_aspas} { printf("Foi encontrada uma String. LEXEMA:
%s\n", yytext); }
{variavel} { printf("Foi encontrada uma Variavel. LEXEMA:
%s\n", yytext); }

.         { /* ignora eventuais caracteres invalidos */ }

%%

int main(void)
{
    yylex();
    return 0;
}

```

5. Conclusão

Esse trabalho foi bem mais interessante do que eu esperava quando li pela primeira vez. A parte que mais me surpreendeu foi perceber que pequenas decisões de ordem das regras mudam completamente o resultado, algo que parece óbvio depois que acontece, mas que você só entende de verdade quando testa e vê a saída errada.

Construir o `lex2.1` do zero, escolhendo os padrões, foi o que mais me fez pensar. Tive que decidir o que fazia sentido reconhecer, montar as expressões regulares e ainda garantir que não havia conflito entre elas, o que me forçou a revisitar os conceitos de classes de caracteres, quantificadores e alternância de uma forma muito mais prática do que só ler sobre eles.

No geral, o trabalho ajudou muito a fixar conceitos da análise léxica, e ficou claro por que essa etapa é importante antes de qualquer análise sintática.